



Programming Exercise 01
Strings and DFA

Developers

- *Royo, Dave Christian*
- *Siosana, Cedric*
- *Tabon, Jeycel Ann*

Tools used

- *Python*
- *Tkinter*

Program Description

The program is a DFA-based string validation application that allows users to load an input file containing strings and a DFA transition table file. The program reads and displays the contents of the selected files, then processes each input string according to the rules defined by the DFA. Each string is evaluated character-by-character, starting from the DFA's start state and following the corresponding transitions. A string is classified as **VALID** if the DFA ends in a final state after processing the entire string; otherwise, it is classified as **INVALID**. The results are displayed in the program's output section and are also saved to a corresponding .out file. The program provides a graphical user interface with controls for loading files, processing the input, viewing the DFA table, viewing the input strings and results, and displaying program status messages. If an invalid DFA file is loaded, it is not accepted, and the program continues using the most recently successfully loaded DFA.

Functions

Since the program was developed by a group of three members, the system was divided into separate modules based on each member's assigned responsibility. Each member developed and maintained a specific module, including the GUI and file-handling module, DFA validation and parsing module, and DFA simulation and integration module. The modules were designed to communicate with one another by passing the required data and calling functions whenever their functionality was needed. This modular approach allowed each member to work independently on their assigned part while allowing the modules to work together as one complete program. After developing the individual modules, the group

integrated and tested them together to ensure that the complete system followed the required program specification.

Module 1: DFA Parser & Validator

This module is responsible for reading and validating the .dfa file and loading the input strings from the .in file. The DFA parser checks the structure and contents of the DFA file before creating the DFA object used by the evaluator.

- `parse_dfa(file_path)` – Reads and validates the .dfa file. It checks the alphabet, state rows, state labels, transition destinations, state types, duplicate states, and the requirement that exactly one start state is defined. If the DFA is valid, the function creates and returns a DFA object containing the alphabet and states. If the file is invalid, it returns an error message instead.
- `load_input_strings(file_path)` – Reads the .in file and returns its contents as a list of strings, with each line representing one input string. Trailing newline characters are removed while blank lines are preserved as empty strings.

Module 2: DFA Simulation

This module is responsible for processing the loaded input strings using the validated DFA and determining whether each string is accepted by the automaton.

- `process_dfa(input_strings, states, alphabet)` – Processes each input string using the DFA, starting from the designated start state and following the corresponding transition for every input symbol. If an input symbol is not part of the DFA alphabet, the string is rejected. After processing the entire string, the function checks whether the current state is a final state and classifies the string as VALID or INVALID. The results are returned in the same order as the input strings.
- `evaluate_strings(dfa_object, input_strings)` – Serves as the modular API wrapper for the DFA evaluator. It receives the validated DFA object and the input strings, extracts

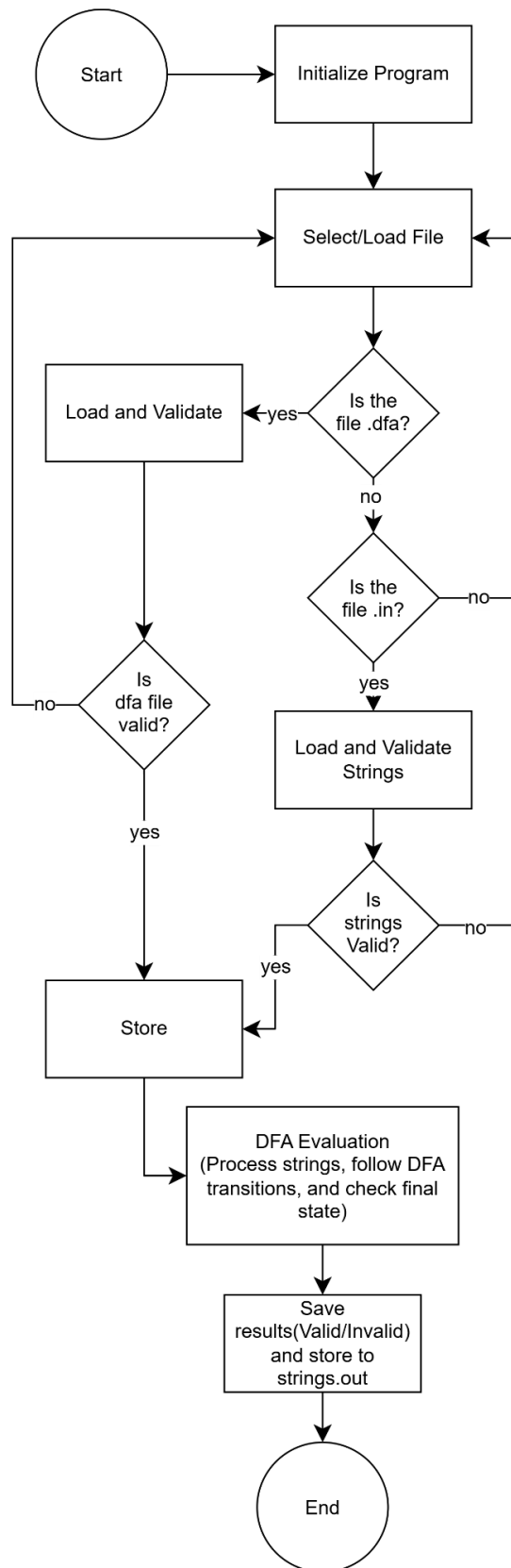
the DFA's states and alphabet, and passes them to `process_dfa()` for evaluation. It returns the resulting list of VALID and INVALID results.

Module 3: GUI & File I/O

This module is responsible for creating the graphical user interface, handling file loading, displaying DFA and input data, processing results, and saving the output file.

- `_configure_styles()` – Applies the visual styles used by the GUI.
- `DFAApp` – Defines and manages the main GUI application.
- `_build_ui()` – Creates and arranges the GUI components.
- `_make_panel()` – Creates styled panels for the interface.
- `_make_button()` – Creates and styles the GUI buttons.
- `_on_load_file()` – Opens the file dialog and determines whether the selected file is a .dfa or .in file.
- `_load_dfa_file()` – Loads and validates a DFA file and displays the transition table when valid.
- `_load_input_file()` – Loads and displays the input strings from the .in file.
- `_render_transition_table()` – Displays the loaded DFA's states and transitions in the transition table.
- `- _on_process()` – Sends the loaded DFA and input strings to the evaluator, displays the results, and saves the .out file.
- `_update_process_button()` – Enables the Process button only when both required files are loaded.
- `_set_status()` – Updates the status message displayed in the GUI.
- `_write_textbox()` – Writes content into the GUI's read-only text boxes.

Control Flow



Work Distribution

Royo:

Royo was responsible for developing the main graphical user interface and handling file input and output. This included creating the Load File and Process buttons, as well as the displays for the DFA transition table, input strings, output results, and program status. Royo implemented the file-loading functionality for .in and .dfa files, allowing files to be selected from any directory and displaying their contents in the appropriate sections of the interface. He also handled writing the validation results to the corresponding .out file and managing the UI state, including enabling the Process button only when both required files had been successfully loaded. In addition, Royo handled status messages and error prompts related to file loading and processing.

Siosana:

Siosana was responsible for developing the DFA validation and parsing engine. This included reading and parsing .dfa files, validating the alphabet, state rows, state labels, transitions, and the presence of exactly one start state. Siosana also developed the internal DFA data structure containing the states, transitions, start state, and final states. The module handles invalid DFA files by rejecting them and ensuring that they do not replace the most recently successfully loaded DFA. Cedric also implemented the loading of .in files and provided the parsed DFA and input data required by the other modules.

Tabon:

Tabon was responsible for developing the core DFA simulation and integration functionality. This included processing each input string character-by-character, starting from the DFA's start state and following the corresponding transitions based on each input symbol. The module determines whether each string ends in a final state and classifies it as VALID or INVALID, returning the results in the same order as the input strings. Tabon also integrated

the DFA evaluation with the Process button so that the loaded input strings could be processed using the currently loaded DFA. In addition, Tabon was responsible for preparing the standalone executable, completing the required program documentation, and assembling the final .zip archive for submission.